Provisiona, Commita e Publica

Infraestrutura como Código com AWS, Terraform e Ansible

Prof. Gustavo Antonio Furquim



INSTITUTO FEDERAL
São Paulo
Campus
Araraquara

O que vamos fazer hoje

Ao final do minicurso, você terá:

- criado uma máquina na AWS escrevendo código, sem clicar no console;
- publicado uma aplicação Java nela, também por código;
- alterado a aplicação, enviado ao Git e visto a mudança no ar.

Ferramentas

AWS Academy Learner Lab, Terraform, Ansible, Git e GitHub.

Cronograma de três horas

Bloco	Tempo
Preparar a máquina e as credenciais	25 min
Seu repositório no GitHub	20 min
Terraform: provisionar a infraestrutura	40 min
Ansible: publicar a aplicação	40 min
Mudar o código e publicar de novo	40 min
Destruir tudo e encerrar	15 min

Por que Infraestrutura como Código?

Pelo console

Cliques que ninguém lembra depois.
Difícil de repetir igual.
Sem histórico do que mudou.
Esquecer recursos ligados custa dinheiro.

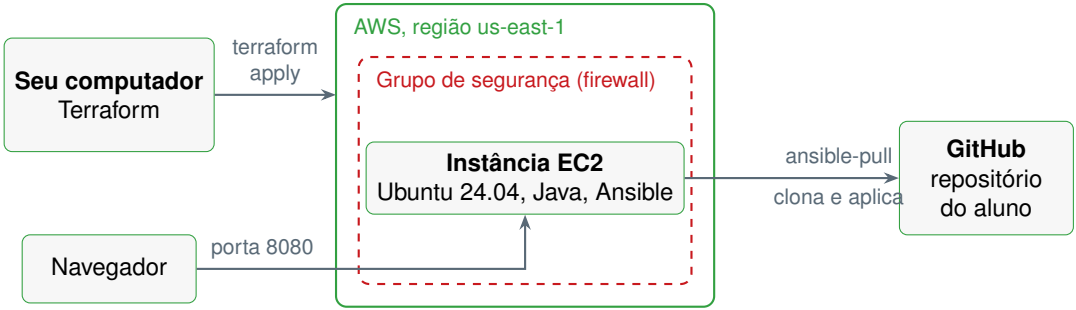
Por código

Arquivos de texto, versionados no Git.
Repetível quantas vezes for preciso.
Cada mudança tem autor e data.
Um comando cria, outro apaga tudo.

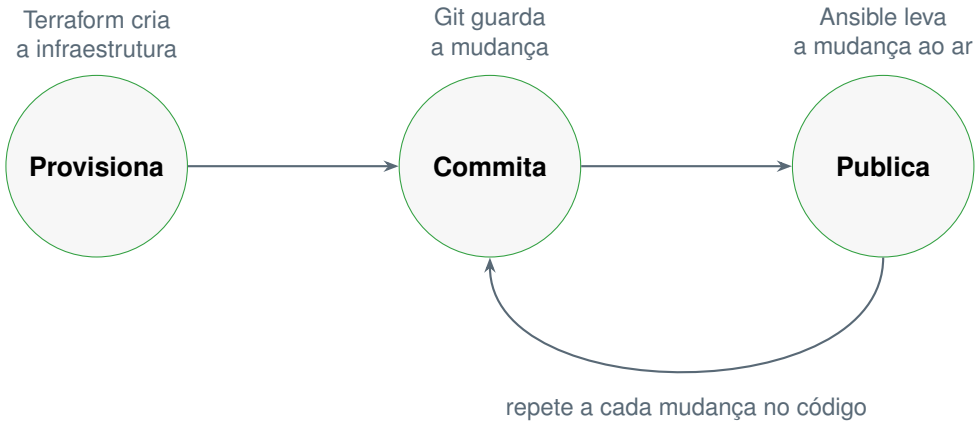
Quem faz o quê

Ferramenta	Papel	Pergunta que responde
AWS	Fornece a nuvem e as máquinas	Onde roda?
Terraform	Cria e destrói a infraestrutura	O que existe na nuvem?
Ansible	Configura a máquina e instala a aplicação	Como está a máquina por dentro?
Git e GitHub	Guardam o código e o histórico	Qual é a versão atual?

A arquitetura do laboratório



O ciclo do laboratório



INSTITUTO FEDERAL
São Paulo
Campus Araraquara

Regras do AWS Academy Learner Lab

Regra	O que significa para nós
Regiões	Só us-east-1 e us-west-2. Usaremos us-east-1.
Chave SSH	Já existe, chama vockey. O arquivo é labsuser.pem.
Credenciais temporárias	Mudam a cada Start Lab e vencem no fim da sessão.
Crédito limitado	Destruir tudo ao final do minicurso.

Instalar as ferramentas (uma vez)

Pelo script de instalação do material do minicurso, que cadastra o repositório oficial da HashiCorp e instala Terraform, Ansible e AWS CLI:

```
bash instalar_terraform_ansible.sh --aws-cli
```

Depois, confira as versões:

```
terraform version
ansible --version
aws --version
```

Credenciais do Learner Lab (a cada sessão)

No AWS Academy: Start Lab, depois AWS Details e Show ao lado de AWS CLI. Cole o bloco copiado em `~/.aws/credentials`.

```
aws sts get-caller-identity
```

Se a resposta trouxer conta e usuário, as credenciais valem.

Cuidado

Não compartilhe nem fotografe o número da conta que aparece na resposta.

Testar a rede antes de começar

Destino	Para que serve	Esperado
registry.terraform.io	Baixar o provedor da AWS	200
sts.us-east-1.amazonaws.com	Autenticar na AWS	302
github.com	Clonar e enviar código	200
portquiz.net:8080	Testar a porta 8080	sucesso

Qualquer código HTTP prova que a conexão chegou. Tempo esgotado indica bloqueio.

Seu repositório: clonar e subir

Primeiro, no GitHub: crie um repositório vazio e público chamado `javadeploy`, sem README.

```
git clone https://github.com/gusfurquim/javadeploy.git
cd javadeploy
git remote set-url origin \
    https://github.com/<seu-usuario>/javadeploy.git
git push -u origin main
```

O repositório precisa ser público para a instância clonar sem senha.

O que é o Terraform

Cria, altera e apaga infraestrutura a partir de arquivos de texto.

- Feito pela HashiCorp, com a linguagem HCL e arquivos `.tf`.
- Funciona com muitos provedores: AWS, Azure, Google Cloud, Docker, Kubernetes.

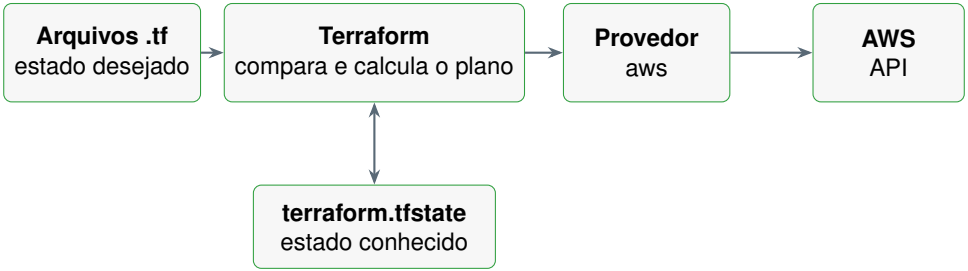
Declarativo

Você descreve como o ambiente deve ficar. O Terraform descobre os passos.

Os arquivos do Terraform

Arquivo	Função	No Git?
`main.tf`	Recursos e fontes de dados: grupo de segurança, instância, imagem	Sim
`providers.tf`	Provedor e versão: aws, região us-east-1	Sim
`.terraform.lock.hcl`	Versões exatas dos provedores usados	Sim
`terraform.tfvars`	Valores das variáveis, como o `my_ips`	Não
`.terraform/`	Provedores baixados pelo `init`	Não
`terraform.tfstate`	Estado: o que o Terraform já criou	Não

Como o Terraform funciona



O plano é a diferença entre o que você quer e o que já existe.

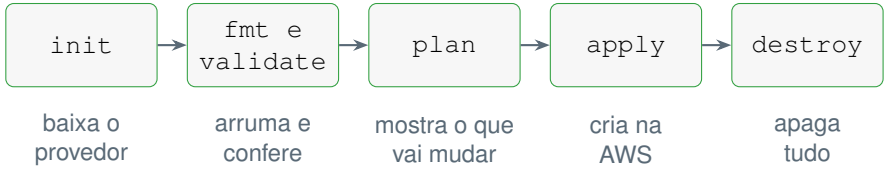
Como se lê um bloco do Terraform

Trecho simplificado:

```
resource "aws_instance" "web" {
  instance_type = "t3.small"
  key_name      = "vockey"
}
```

`resource`	palavra que declara um recurso
`"aws_instance"`	o tipo do recurso: uma máquina EC2
`"web"`	nome local, usado para fazer referência a ele
entre chaves	os argumentos que descrevem como ele deve ser

O fluxo de trabalho do Terraform



O `plan` é o primeiro comando que fala com a AWS.

O que a configuração descreve

Peça	Papel
provider aws	Diz com quem conversar: AWS, região us-east-1
data aws_ami	Descobre a imagem do Ubuntu 24.04
aws_security_group	Firewall: SSH só dos IPs do campus, porta 8080 aberta
aws_instance	A máquina t3.small, com a chave vockey e um script de inicialização
output	Mostra `ssh_command` e `app_url`

Seu IP: a variável my_ips

Crie o arquivo `terraform.tfvars` na pasta `terraform` (ele fica fora do Git):

```
my_ips = ["200.100.50.25/32", "200.100.50.26/32"]
```

- o campus sai para a internet por dois endereços: coloque os dois;
- `/32` significa um único endereço;
- seu IP aparece em `curl http://portquiz.net:8080`.

Antes de digitar yes, leia o plan

Sinal	Significado
+	Será criado
~	Será alterado no lugar
−	Será destruído

Na primeira execução esperamos só criações: *Plan: 2 to add, 0 to change, 0 to destroy*.

Hábito

Um destroy inesperado é motivo para parar e perguntar.

Entrar na instância

```
ssh -i labsuser.pem ubuntu@<ip-publico>
cloud-init status --wait
ansible --version && java -version && git --version
```

O comando SSH pronto aparece no `ssh_command` do apply. Se o SSH reclamar da chave, use `chmod 600 labsuser.pem`.

O `cloud-init` termina de instalar Git, Ansible e Java; espere *status: done*.

O que é o Ansible

Automatiza a configuração do que existe dentro das máquinas.

- Instala pacotes, copia arquivos, cria usuários, gerencia serviços.
- Sem agente: basta SSH e Python nas máquinas gerenciadas.
- As instruções ficam em arquivos YAML, fáceis de ler.

Terraform e Ansible

O Terraform cria a máquina. O Ansible a deixa pronta para a aplicação.

Os arquivos do Ansible

Arquivo ou pasta	Função
`local.yml`	O playbook: as tarefas que o `ansible-pull` executa
`templates/webapp.service.j2`	Modelo Jinja2 da unidade do systemd
Inventário	Lista das máquinas; aqui vai na linha de comando: `-i localhost,`
`ansible.cfg`	Configurações opcionais do Ansible
`~/.ansible/pull/`	Cópia do repositório feita pelo `ansible-pull` na instância

YAML: o formato dos playbooks

```
servidor:
  nome: webapp
  porta: 8080
  pacotes:
    - git
    - ansible
```

`chave: valor` forma um mapa;

`- item` forma uma lista;

a indentação mostra quem pertence a quem.

Legível por pessoas e usado também em Docker Compose, Kubernetes e GitHub Actions.

YAML, JSON e XML: comparação

YAML

```yaml
servidor:
  nome: webapp
  porta: 8080
  pacotes:
    - git
    - ansible
```

JSON

```json
{
  "servidor": {
    "nome": "webapp",
    "porta": 8080,
    "pacotes": [
      "git",
      "ansible"
    ]
  }
}
```

XML

```xml
<servidor>
  <nome>webapp</nome>
  <porta>8080</porta>
  <pacotes>
    <item>git</item>
    <item>ansible</item>
  </pacotes>
</servidor>
```

	YAML	JSON	XML
Comentários	Sim	Não	Sim
Uso típico	Configuração	APIs	Documentos

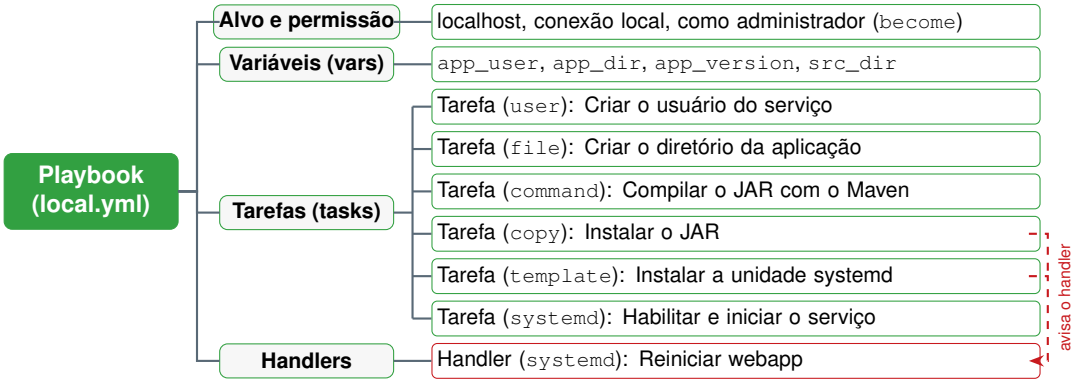
Cuidados ao escrever YAML

Assunto	Regra
Indentação	Só espaços, nunca tabulação. Dois espaços por nível é o costume.
Dois pontos	Precisa de um espaço depois: `nome: webapp`.
Lista	Traço e espaço: `- item`, todos alinhados.
Comentário	Começa com `#` e vai até o fim da linha.
Caracteres especiais	Texto com `:` ou `#` vai entre aspas.
yes, no, true, false	Podem virar valor lógico, não texto.

O vocabulário do Ansible

Termo	O que é
Inventário	A lista das máquinas em que o Ansible vai agir
Playbook	O arquivo YAML com o roteiro do que deve ser feito
Tarefa	Uma única ação dentro do playbook
Módulo	O código pronto que executa a tarefa: instalar pacote, copiar arquivo, gerenciar serviço
Handler	Tarefa especial que só roda quando outra avisa que houve mudança
Template	Arquivo com variáveis, em Jinja2, preenchido ao ser copiado

A árvore do local.yml



INSTITUTO FEDERAL
São Paulo
Campus Araraquara

Como se lê uma tarefa do Ansible

Trecho simplificado:

```
- name: Garante o servico ativo
  ansible.builtin.systemd:
    name: webapp
    state: started
```

`name`	descreve a intenção, para humanos lerem
`systemd`	o módulo que faz o trabalho
`state: started`	o estado desejado: o serviço deve estar iniciado

A ideia central do Ansible: idempotência

Descrevemos o estado desejado, não uma lista de ações.

Primeira execução

O estado difere do desejado.
O Ansible corrige e mostra changed.

Segunda execução

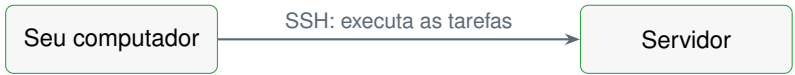
O estado já é o desejado.
Nada é refeito e aparece ok.

Por isso repetir o playbook é seguro.

Idempotente: aplicar uma vez ou várias vezes dá o mesmo resultado, como em $f(f(x)) = f(x)$.

Push ou pull?

Ansible comum (push)



ansible-pull (pull)



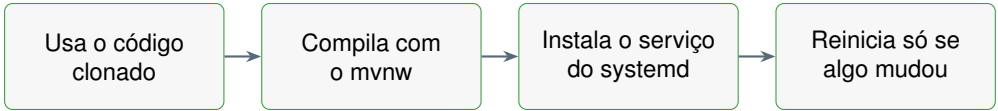
O comando que publica a aplicação

```
ansible-pull \
    -U https://github.com/<seu-usuario>/javadeploy.git \
    -i localhost, local.yml
```

`-U`	de onde clonar o repositório
`-i localhost,`	o alvo é a própria máquina; a vírgula é obrigatória
`local.yml`	o playbook que será executado

Rode dentro da instância, ou de fora pelo SSH.

O que o local.yml faz



Testar a aplicação

Dentro da instância

```
curl http://localhost:8080/api/hello
```

Se algo falhar

```
sudo systemctl status webapp
journalctl -u webapp -n 30
```

No navegador: o `app_url` do apply, na porta 8080.

Plano B, se a porta 8080 estiver bloqueada: túnel SSH

```
ssh -i labsuser.pem -L 8080:localhost:8080 ubuntu@<ip-publico>
```

Depois abra `http://localhost:8080` no seu navegador.

Mude o código e publique de novo

No seu computador, edite o `index.html` ou o `HelloController.java` e envie:

```
git add .
git commit -m "Altera a mensagem"
git push
```

Na instância, rode o mesmo `ansible-pull` de antes e recarregue o navegador.

Quando algo dá errado

Sintoma	Causa provável	O que fazer
ExpiredToken	Credenciais vencidas	Copie o bloco AWS CLI de novo
SSH com tempo esgotado	Seu IP fora do `my_ips`	Confira os IPs e rode o apply
Permission denied no push	Senha no lugar do token	Use o token como senha
Página não abre	Porta 8080 bloqueada	Túnel SSH com `-L`
Serviço não sobe	Erro no código Java	`journalctl -u webapp -n 30`

Encerrar: destruir antes do End Lab

Na pasta `terraform`:

```
terraform destroy
```

Digite `yes`. O resultado esperado é *Resources: 2 destroyed*.

Ordem importa

Só depois do destroy, clique em End Lab no AWS Academy.

Recapitulando

Provisiona

O Terraform criou a máquina e o firewall.

Commita

O Git guardou cada mudança do código.

Publica

O Ansible colocou a aplicação no ar.

Repetir qualquer passo é seguro, e tudo pode ser reconstruído a partir dos arquivos do repositório.

Para ir além

- Contêineres: Terraform e Ansible subindo imagens Docker em vez de um JAR.
- Segredos: guardar senhas com o Ansible Vault.
- Publicação automática a cada push, com o GitHub Actions.
- Um segundo servidor, com o banco de dados separado da aplicação.

Dúvidas?

INSTITUTO FEDERAL
São Paulo
Campus
Araraquara